



ITCY354: Secure Software Engineering

DR. Alya Sayed Husain Alkamel

First Semester 2025-2026

Systems for modelling and simulation :

Cyber Attack Simulation Platform

Aljazi Ali Almujaaddam - 202202227

Dana Hussain Alhayki - 202204341

Muneera Yusuf Almehaiza - 20226145

Sec 01

Submission Date: November 16, 2025

Table of contents

Table of contents.....	2
Problem Definition.....	3
Project Objectives.....	4
Significance and Impact.....	5
Brief Literature Review.....	7
Scope and Limitations.....	9
Software Process Model.....	12
Project Management Plan.....	14
Risk Management Plan.....	19
Table 1:Risk summarization table.....	19
Functional and Non-Functional Requirements.....	23
Security Requirements (Using the SQUARE Methodology).....	28
Table 2: Prioritization of Security Requirements.....	35
System and Design Modeling.....	38
• High-Level System Architecture.....	38
• Use Case Diagram.....	39
Figure 1 Use case diagram:.....	39
• Class Diagram.....	40
Figure 2 Class diagram :.....	40
• Data Flow Diagrams (DFD).....	41
Figure 3 Data Flow Diagram level 0 :.....	41
Figure 4 Data Flow Diagram level 1 :.....	42
• Interface Design (Figma Prototype Screens).....	43
Figure 5 Login with 2FA :.....	43
Figure 6 Dashboard (simulation overview) :.....	44
Figure 7 Scenario setup (select attack, defenses) :.....	44
Figure 8 Live monitor (traffic chart, IDS alerts) :.....	45
Figure 9 Reports view and export (Admin) :.....	45
Figure 10 Reports view and export (Student) :.....	46
• Secure Architecture Principles in Design.....	46
Threat Modeling and Security Analysis.....	49
Table 3: Threat modeling and security analysis table.....	50
Evaluation and Discussion.....	57
Future Work.....	60
References.....	64

Problem Definition

The current organizations and educational establishments have a problem of educating people to manage cyber attacks without exposing real systems at risk. The cybersecurity domain lacks the safe, realistic simulated environment to be practised in, unlike the other area like aviation or medicine where there is the use of simulators (Cyber Range Project Team and the NICE Community Coordinating Council, Cybersecurity Skills Competitions Community of Interest, 2023, 6). Due to heightened and more advanced cyber threats (such as DDoS attacks and ransomware), a platform designed to recreate cyber attacks under a controlled environment is urgently required. The gap to be covered by this project is that there is no available platform of cyber attack simulation that users can experiment and learn cyber attacks (and defensive strategies) within a safe and legal setting (Cyber Range Project Team and the NICE Community Coordinating Council, Cybersecurity Skills Competitions Community of Interest, 2023, 6).

This is also a matter of concern in terms of security since in the absence of these training platforms, people are not always ready to respond to real attacks, which results in slowness or lack of response. Organizations may enhance their security position and dependability of their systems by being able to offer a dedicated simulation atmosphere: trainees can exercise detecting and mitigating attacks without risking live networks. Regarding software reliability, the software platform should be written in a way that prevents malicious code (so that any malicious code escaping into the real world does not do any harm) and which is not fragile even when subjected to severe attack testing. In brief, the project addresses the twofold issue of cybersecurity training (to develop human knowledge and readiness) and the fact that the training tool must be structured with a high degree of security to avoid a new threat.

Project Objectives

The Cyber Attack Simulation Platform prototype has straightforward aims - each one can be tracked easily while sticking to safe coding practices.

Simulating Common Cyber Attacks:

The system's designed to mimic big online threats - say, a flood-style DDoS strike or a ransomware breakout. Besides those threats, there'll be shields like a firewall paired with an Intrusion Detection Setup. That combination lets you check how network-wide and device-specific attack situations play out - all without real-world risks.

Safe and Secure Architecture:

security is embedded into the blueprint right away. The sim runs inside a locked-off zone, cut off completely from core operations so fake attacks stay contained. Each piece sticks to basic safety rules - think minimal access, layered protection. Aim's clear: let Users test freely, but keep things harmless and out of bad hands.

Integrated Security Requirements Engineering:

The project follows the SQUARE approach to figure out which security needs matter most right at the start. That way, key elements like user verification, permission settings, or keeping data safe get sorted out upfront rather than tacked on after. Each of these demands is spelled out plainly and tied directly to how the system gets built.

Comprehensive Modeling and Design:

Before any code is written, we'll sketch out clear visuals - like use case charts, class layouts, data movement maps, along with sample screens - to show how everything works. These drawings give a preview of the system's actions plus where safety controls go. We'll make sure to build at least one example from every category, check it over, then sign off if it lines up with what's

needed. Doing this upfront catches flaws in structure or protection before they become bigger problems.

User-Friendly Training Interface:

A basic version of the app's layout - built in Figma or something alike - shows between three and five key pages walking people through each step: pick a cyberattack type, build safeguards, start test runs, then check outcomes. Instead of clutter, it keeps things clear and easy so anyone from new students to users learning online won't get stuck. On top of that, safety features are built into how it looks and works, such as sign-in checks and double-approval before dangerous moves, pushing safer behaviors by default.

Threat Analysis and Mitigation:

The crew's using STRIDE threat modeling to spot potential system threats, while also checking how they might play out. When a risk pops up, fixes either get suggested right away or built into the layout from the start. Say, dealing with fake access tries or fending off service crashes aimed at the platform - those kinds show up here. Findings land in a threat model chart that lays out what's been done about every serious problem uncovered.

Once these goals are met, the platform'll offer a clear, well-protected setup for cyber attack simulations - not just useful for training but built with safety from the start.

Significance and Impact

Target Users and Stakeholders:

The primary users of this platform are cybersecurity students looking for hands-on practice to go alongside their studies, along with IT security users like SOC analysts or incident handlers wanting to sharpen their day-to-day abilities, plus businesses and public sector groups that want ongoing team training using lifelike attack simulations. Teachers and coaches in cyber fields matter too because they can use the system to build and run labs or drills more effectively. In

broader terms, the entire cyber world gains value since the setup might later act as a proving ground for testing fresh protection methods or tech ahead of live implementation.

Expected Benefits:

For learners just starting out, this space gives a do-it-yourself setup where ideas meet actual use. It supports risk-free testing of offensive moves along with protective strategies, building skill and self-assurance over time. Firms running mock drills tend to respond quicker during crises while boosting teamwork on the ground. Studies show squads using cyber ranges manage threats much faster compared to groups skipping these sessions.

The platform helps shrink the cybersecurity skill shortage by training people through hands-on practice in simulated attacks, instead of passive study. Plus, it builds a mindset of staying ahead of threats - since groups regularly run drills in risk-free settings, tweaking plans and getting better prepared gradually.

Impact on Secure Software Practice:

Besides teaching skills, this effort boosts safe coding habits by showing ways to weave protection into every phase of building software - not just at the end. This system acts like a real-world example, walking through safety steps starting with defining needs using SQUARE, then moving into planning and spotting risks early. Instead of adding safeguards later, it builds them in from day one, pushing the idea that security should shape how code is made right from the start.

In real use, the platform might double as a protected space where different software gets tested. When new apps are set up inside the simulated zone, it's possible to see how they hold up when attacked - giving useful feedback ahead of live rollout. That boosts not just dependability in programs, but strengthens development workflows with built-in security.

If stretched past the early test phase, this setup might grow into an ongoing security check-up and learning tool - shaping how teams handle safe software practices. What makes it matter is how it offers a clearer, organized method to mimic hacking attempts for teaching or job prep, at the same time showing what solid security design can achieve.

Brief Literature Review

Related Systems: Cyber attack simulation platforms known as cyber ranges exist in both industrial and academic environments. Cloud Range operates as a commercial cyber range service which delivers live-fire cybersecurity training to organizations. The Cloud Range platform enables teams to practice incident response for realistic attack scenarios including ransomware outbreaks and network intrusions within a secure isolated range environment (Guest User, 2023). The platform teaches both technical competencies and team-based communication skills for incident response while offering practical training to professionals who need to close the cybersecurity talent gap (Guest User, 2023).

The KYPO Cyber Range Platform stands out as a notable academic and open-source system which Masaryk University developed. The KYPO cyber range operates as an open-source platform which enables educational institutions and organizations to develop practical cybersecurity training exercises through its cloud-based infrastructure (Masaryk University, 2022). The system uses OpenStack virtualization to create detailed network simulations which function as digital duplicates of IT systems and it offers free access to address the shortage of cybersecurity professionals through enhanced training accessibility (Masaryk University, 2022). The field contains two types of cyber ranges: government and industry-operated proprietary ranges and specialized simulation tools which focus on specific attack types such as DDoS or malware analysis.

Comparison and Limitations: The current platforms show that cyber attack simulation works effectively yet they have specific restrictions. The comprehensive features of Cloud Range come with high expenses and restricted customization options for end-users. Organizations need to

work with the provider to execute exercises because the pre-designed scenarios focus on general use cases. The open-source KYPO platform provides free access but users need to handle complex technical requirements and maintain substantial cloud infrastructure and specialized knowledge for deployment.

The current range of systems focuses on exercise content and scoring systems instead of showing how their platform security features operate. The documentation about these systems explains their functional capabilities but fails to reveal their internal software security mechanisms. Our project focuses on solving this specific problem. The current range of scenarios in existing systems provides wide coverage but lacks detailed analysis of combined attack-defense scenarios; users cannot select specific IDS configurations for ransomware scenarios.

How Our Project Addresses These Limitations:

The "Cyber Attack Simulation Platform" prototype is a secure architecture and requirement-focused product with minimal emphasis on the documentation of existing solutions. Since we focused on secure software engineer practices-like the SQUARE process and STRIDE analysis-according to our project, the platform will be one that deals with threats and vulnerabilities systematically from the very beginning-a method which might be missing or of a proprietary nature in other systems. Functionally, our prototype is a compromise between reality and user-friendliness.

Our heavy plan that requires cloud infrastructure could be realized on a smaller scale-e.g., a university lab setting or a single high-performance server-to simulate certain scenarios like DDoS and ransomware attacks. Therefore, it becomes less expensive and the problem of limited resources is solved for small organizations and educational institutions-we think that the future implementation might use containerization for easy deployment, for example.

Also, we are planning the platform to be full of features and highly customizable; stakeholders-instructors or admins-through an intuitive interface can generate their own attack scenarios and defensive measures while in the case of the commercial range, vendor support is

usually needed. Besides, we secure that security is an integral part of the platform-closure of simulations, role-based access, etc.-which means that our very design can be looked at and verified by the community-thus contributing to secure design knowledge for such systems.

In brief, our project is a secure, educational prototype that is cheaper, community-friendly in terms of its design, and focused on demonstrating the interactions between attacking and defending with detailed security in mind, inspired by existing cyber ranges.

Scope and Limitations

Scope: The project includes the design of a prototype cyber range platform that can simulate cyber-attacks and defenses without actual deployment. It specifically includes the following: -

Selected Attack Simulations:

We are focusing on two representative attack scenarios: one is a network-based attack, such as a DDoS flood, and the other is a host-based attack, such as a ransomware infection on a virtual machine. These scenarios cover different threat vectors and require different defenses; they give a broad view of the capabilities of the platform.

Defensive Measures:

The design will integrate common defensive components into the simulation. For example, a virtual firewall will be part of the network scenario to filter or throttle traffic during a DDoS simulation. Similarly, in the host scenario, an Intrusion Detection System (IDS) component will monitor for malicious behavior, like the signatures of ransomware activity. The platform will simulate how these defenses respond, such as how the firewall mitigates a DDoS or how the IDS alerts on ransomware, and provide feedback to the user.

User Roles & Interface:

The system has the role of a Trainee/User conducting simulations and an Administrator who manages the configurations of the platform, such as adding new scenarios or managing user

access. We will design an interface for the workflow of a trainee: logging in, selection of a scenario, running the simulation, viewing results. Then we outline the admin capabilities. Interface design via Figma prototypes shall be created for ~5 core screens in scope to demonstrate the user experience.

Secure Software Process:

In terms of the activities to be performed for the project, the scope will cover requirements engineering, design modeling, and security analysis. It will produce artifacts like use case diagrams, class diagrams, DFDs, a security requirements document via SQUARE, and a threat model. We are including all these to make sure the final design is well-documented with a security-focused mindset.

High-Level Architecture & Data Flows:

The project will deliver a high-level system architecture description, identifying key modules-e.g., simulation engine, environment manager, data stores-and how data flows between them during an attack simulation. We describe how the platform orchestrates an attack scenario from start to finish, including any logging or reporting mechanisms.

Risk and Mitigation Planning:

The project recognizes that this is only a prototype design and thus not to be implemented. However, a risk management plan for both project execution and possible technical risks in the design was included, which is within scope.

Limitations (what's left out or restricted): Some parts aren't covered because this is just a prototype made for school work. Instead of full development, it's only about planning - no real code gets written, no hacking tools plugged in, nor any active viruses tested; also, nothing runs on live servers. Because there's no built version, things like speed tests or feedback from users won't show up here. On top of that, the range of threats and protections looked at stays narrow - just DDoS and ransomware as dangers, along with firewall and intrusion detection as shields. This setup doesn't try to handle every possible cyber threat you might face.

Take this: right now, our model doesn't cover web-based threats - say, things like SQLi or cross-site scripting - nor does it include safeguards such as anti-virus tools or security monitoring systems. These might come later, yet they're left out for now. Here's the setup - it runs under tight conditions where every test happens in an isolated space cut off from outside networks. That means fake malware won't reach actual internet servers during trials; instead, those actions are mimicked internally. No live connections allowed.

Why? Safety first - and any working version down the line would need to follow the same rule. Some actions in real life could get boiled down or made easier to handle - say, swapping real malware with fake data that acts like ransomware encryption. We're picturing just a handful of people using it at once, nothing heavy-duty, since this setup works best for test runs or school labs.

Handling hundreds at the same time? That's not part of the plan right now. User controls stay minimal; no links to company login systems or layered access rules are included here. The whole thing counts on tools you can already find - VMs or containers - to keep things separated and running smooth. No need to build everything bottom-up, like crafting a new virtual machine manager ourselves; we'd rather use what's already around, such as VirtualBox, Docker, or any standard cloud-like space.

This setup leans on outside tools - the actual isolation and speed you get later will hinge on what's underneath. Because school schedules squeeze us, a few nice-to-have bits got cut. Take full feedback tracking - rating how users handle scenarios - or built-in coaching during drills, like smart tips or shifting attack styles; they're just not here. Plus, mapping threats and pinning down needs comes from what we know and papers we read, not live talks with pros or testing the system under real hacking stress - that'd take more hours and cash than we've got.

That gap means some sneaky flaws might slip through, and yeah, we see that. Even though the sim tries to feel real, parts are dialed back. A DDoS hit, say, shows up as fake log entries and alerts instead of actually flooding connections. Ransomware impacts could appear like file

scrambling actions inside logs, instead of locking real data on storage drives. Such shortcuts are constraints needed for early models - since handling harmful viruses isn't practical now - but improvements may come later down the line.

Defining the boundaries straight up keeps the project on track, so it stays doable while laying out what this early version handles - along with what's off-limits. Stuff beyond these limits gets flagged now, just in case it matters later.

Software Process Model

Building this thing meant picking a good way to handle software steps, 'cause safety matters big time. One option was step-by-step linear work, another spun around with loops - Agile came up too - but in the end, we went with the looping approach 'cause it worked best.

Waterfall Model: This method moves step by step in a straight line, going through separate stages - like figuring out needs, creating designs, building the product, checking for bugs, and ongoing updates. One plus side? It makes planning easy and sets obvious checkpoints along the way. Still, it counts on knowing every detail from day one, expecting almost no shifts once work starts. When security's a top concern, that lack of flexibility backfires - say a safety need gets overlooked at first; chances are it won't pop up until final checks - or maybe never - leading to major rework down the road. Because our project's still figuring out how to safely mimic attacks - plus threats keep shifting - a strict Waterfall setup would be way too rigid. On top of that, Waterfall doesn't build in clear moments for spotting risks, something security absolutely needs.

Agile Model (like Scrum): These methods prioritize step-by-step progress, input from users, also flexibility when things shift. With this style, we'd develop the system bit by bit - using short cycles - and improve it as we go. It works well when needs evolve or fast responses matter. While rolling out the platform, we might use Agile to launch a simple version fast, then layer on extras over time. Still, plain Agile doesn't lay out clear steps for handling risks or security - those need to slot in via methods like DevSecOps. Running several complete Agile cycles can be tough in a brief research-based project. Even more relevant, we're not writing code; instead, we're

building design documents and assessments, so Agile's obsession with working software fits poorly - we're turning in papers and diagrams. What worked better was a flow that highlights checking risks and validating designs every step, not just pumping out features fast.

Spiral Model: For this project, we went with the Spiral approach. First laid out by Barry Boehm, this process focuses on tackling risks step by step through repeated cycles. Picture a winding path where each full loop hits four stages - planning stuff like goals, options, limits; then checking possible problems; followed by building and testing the actual software; finally stepping back to assess what's done and map out the next round. One big reason this model fits so well is how it makes risk review part of every single cycle (Talreja, 2025).

This fits our project well, given that handling security risks matters just as much as meeting functional needs; every turn of the spiral model lets us rethink requirements and design while weighing potential dangers. Take the first loop - we collected needs (like security targets) while spotting broad risks (say, unsecure simulations). Moving forward, during the following phase, we built design drafts and examined deeper threats (using STRIDE), which led us back to update requirements (tweaking security rules when necessary).

This back-and-forth tweaking went on steadily, cutting down guesswork while keeping things lined up with our safety goals. The Spiral model helped tackle risks through quick mock-ups - if a design choice felt shaky, we'd sketch a rough version or walk through a scenario to test how well it held up security-wise or worked in practice, then keep it or fix it based on what we found.

Basically, Spiral handed us a clear setup to bake security steps into every phase. By checking dangers often and going over drafts as we went, we avoided nasty surprises near the finish line (*Spiral Model: The Risk-Minimizing Model for Software Projects*, n.d.). On top of that, Spiral bends easily - it pulls in pieces from other methods (some even call it a "model of models") - so one loop might act like a small waterfall for certain features, while another shifts into rapid loops similar to Agile for different parts (*What Is Spiral Model in Software Engineering?*, 2025).

The Spiral model made sense here - security mattered a lot, so we needed constant risk checks; Spiral builds that into every cycle, unlike Waterfall, while Agile would've meant bolting on extra risk steps later. Since our grasp of needs and threats shifted over time - thanks to research and modeling work - going in circles actually helped; Spiral let us tweak requirements after threat analysis, something clunky to pull off in a straight-line Waterfall setup.

Even though we didn't code up prototypes, we still used mock-ups like rough diagrams or scenario sketches to test ideas; that lines up with Spiral's push for using prototypes as risk shields (*What Is Spiral Model in Software Engineering?*, 2025). Spiral's focus on checking back with stakeholders worked well for our school setting where feedback from instructors shaped progress; after each chunk of work - say, nailing down early specs or drawing diagrams - we paused to ask if things were on track or needed redirection next time. Bottom line, Spiral's focus on guiding decisions by risk is often suggested when dealing with unfamiliar tech or murky conditions, which matched our situation.

Building a cyber range system - something we haven't tackled before in class - comes with unknowns, like whether our isolation method might fall short. The spiral approach helps lower the chance of things going wrong by constantly spotting and handling these doubts along the way (*Spiral Model: The Risk-Minimizing Model for Software Projects*, n.d.).

So, sure, Waterfall gave order, whereas Agile brought flexibility - yet the Spiral approach bundled step-by-step progress along with automatic risk checks, which turned out perfect for our secure software effort. It made certain security wasn't skipped, but instead reviewed over and again in an organized way, boosting trust in how the end product took shape.

Project Management Plan

To meet the deadline, we laid out a plan - figuring roles, timing, along with ways to keep communication going. That structure helped us stay focused, ensuring each step was handled well, from collecting requirements through designing mockups to reviewing results, while letting collaboration work naturally.

Timeline & Milestones:

We broke the project into separate phases stretched across 11 weeks, depending on how the classes ran - each phase had specific goals along with clear outcomes.

Phase 1: Initiation & Planning (Weeks 1–2)

We started by pulling the team together, prepping the area - after that, nailing the title "Cyber Attack Simulation Platform." What were we aiming at? Knocking out the Project Proposal; this spelled out our focus and reasons, finishing it all before week two wrapped.

Phase 2: Requirements Analysis (Weeks 3–5)

In this stage, we gathered every requirement tied to function, performance, or safety - writing each one clearly. Even though Week 3 focused on general goals, by Week 4 we shifted toward narrowing security aspects through the use of SQUARE methods. At the same time, earlier research gave direction and influenced our choices. Come Week 5, finishing the initial complete draft of the Requirements Spec became the key milestone.

Phase 3: Design & Modeling (Weeks 6–8)

This phase focused on setting up the system framework alongside basic visual designs. From hand-drawn scenarios paired with short descriptions in week six, progress shifted toward completing class diagrams along with a loose structural draft by week seven, followed by finalizing data flow visuals - covering broad overviews and deeper layers - by week eight. Meanwhile, work continued refining an interactive prototype built in Figma, compiling viewable mockups for essential screens. Check-ins among team members occurred often to keep components aligned.

Goal? Getting all design plans combined with functional screen versions wrapped up ahead of week nine's start.

Phase 4: Security Analysis (Week 9)

We did threat modeling alongside risk assessments using STRIDE to find possible attacks. Whenever threats appeared, we judged their likelihood, impact, and how to lower the risks instead. Once we went through the findings, we made small changes to the design for better defenses. In the end, we handed off both the Threat Model and Risk Plan right before Week 9.

Phase 5: Documentation & Review (Weeks 10–11)

In the final phase, we wrapped up the entire document - double-checked each section closely - so that everything from start to finish matched what we needed. Every key point linked directly to how things were built; whenever problems popped up, solutions had been figured out ahead. By Week 10, main content was set, whereas Week 11 shifted toward refining details, including slides.

Major milestones? Hitting send on the complete report by mid-Week 11 (November 16, 2025), followed by acing the last presentation in Week 12.

Team Roles & Responsibilities:

One person took charge of certain tasks, then everyone met up at major points to check things still fit well together. We worked alone most times - yet swapped feedback often so the full effort made sense all through. Each did their part without stepping back from shared checks now and again.

Aljazi Ali Almujaad – Project Management, Evaluation & Future Planning

Starting off strong, Aljazi steered the whole effort right from day one - goals stayed sharp, doable, without fluff. She put together the Project Management Plan, which kept tasks lined up and deadlines hit when they should've been. Beyond just syncing things up, she handled writing both Evaluation and Discussion parts, breaking down test findings along with what teammates had to say. When it came to Future Work, she chipped in ideas that actually made sense for growing and upgrading the system down the road.

Besides drafting the text, she took care of how the report looked - tidying up layouts, shaping the last slideshow bits so everything felt unified, never sloppy. Because she spotted small stuff fast and kept things sorted tight, the crew moved forward without hiccups, gliding from one stage to the next without snagging.

Dana Hussain Alhayki – Requirements, Design & Interface Modeling

Dana worked on how the system would be built and look. First off, she spelled out the core issue - what it was about, why it mattered, why it made a difference. Then came laying down what the system had to do, along with how it should behave behind the scenes, making sure real-world demands turned into actual working parts.

She stepped up on the UI side, building clickable previews in Figma - covering key areas like sign-in, home view, and threat test features. On top of that, she mapped out the Class Diagram to show how different parts connect logically. Teaming tightly with Aljazi, Dana helped shape System and Design Modeling, making sure structure visuals matched security goals while interface layouts balanced ease-of-use with safety demands.

Muneera Yusuf Almehaiza – Research, Methodology & Security Analysis

Muneera handled the groundwork for the project's research and tech side. While she drafted the Brief Literature Review, she pulled together major findings about secure system setups along with AI-driven flaw handling. Because of her input into the Scope and Limitations part, it became clearer where the project would focus - also spelling out which goals were doable versus beyond reach.

She picked out a Software Process Model, then used it to walk the team step by step through every stage of building the product. On top of that, Muneera wrote up a Risk Management Plan - this listed possible problems while mapping ways to reduce their impact. What she spent a lot of time on was shaping Security Requirements, working with the SQUARE approach before moving into Threat Modeling along with deeper Security Analysis. Because of her efforts, security features got built right into how the system actually works.

Collaboration and Review Process

Crew worked together from start to finish. People kept passing along progress reports, checking teammates' work, while tossing in helpful suggestions now and then. Take Dana with Muneera - they went through Aljazi's planning docs just to confirm what was doable tech-wise. Then again, Aljazi plus Muneera looked over Dana's layout sketches and flowcharts so things lined up right with safety rules but also made sense for users.

This ongoing loop of checking each other's work kept things accurate, steady, plus clear in tone throughout every part of the project. Because we used what each person did best while staying in touch regularly, the end result came together smoothly - organized, solid, showing everyone's contribution.

Communication Methods:

Because the work involved many connected pieces, clear chatting was key. To stay looped in, we used various tools instead of just one, plus quick updates each day:

We get together every Thursday - now in person, now over Zoom - to check progress while working out design decisions side by side.

WhatsApp Group: Handy for quick messages, sharing updates, or passing short bits of news every so often.

Google Drive became our main place for storing files, drawings, or first drafts. When working together on text, we used Google Docs - meanwhile, presentations were done through Google Slides.

Muneera watched the updates on her own. The drafts began in Google Docs - people added comments until it was locked down.

Trello Board: Tracked tasks by phase, showing progress at a glance - also made it clear which person did which job.

Sticking to the plan got every deadline met on time, yet made sure no one fell behind. Because tasks were clear, steps stayed in order, so check-ins happened often - our progress remained steady all through.

Risk Management Plan

We found a few possible issues touching tech, day-to-day work, and safety stuff. For every one, we looked at how likely it is to happen along with what kind of effect it could have on the project - or the setup - then came up with ways to reduce the danger. According to Table1, our main concerns are listed with descriptions, along with the mitigation strategies to mitigate their risks.

Table 1:Risk summarization table

Risk Description	Likelihood	Impact	Mitigation Strategy
Team member unavailability (e.g., illness or other obligations causing a key member to fall behind on tasks).	Low	Medium	Get teammates familiar with key jobs - this way someone else steps in when required. Keep extra time around deadline dates just in case things run late. Share updates often; that helps catch slowdowns fast while there's still room to shift work around.
Scope creep or changing requirements during the project (adding new features or scenarios beyond the initial plan).	Medium	Medium	We'll use what's already known - like textbooks or studies on how attacks work - instead of building everything new. Where we can, make the model easier by showing outcomes without tracking every single data detail, just enough to

			keep things clear. Talk to someone who knows the topic - maybe a teacher - to check our method actually works. Instead of guessing, get feedback from a person familiar with this stuff. A quick chat could save time later. Reach out before moving forward. Also, set aside more hours during planning to tweak tricky sections like DDOS building + crafting a layout look at it closely, especially these parts.
Technical complexity in simulating attacks leads to design errors or delays (for instance, difficulty in accurately modeling a DDoS attack pattern or ransomware behavior).	Medium	High	changes to control creep.
Security design flaw not identified in the platform (e.g., an oversight that the simulated malware could exploit the platform, or a requirement like	Low	High	Carry out detailed safety checks now and then - try this method while mixing in that one Use STRIDE to check the setup against every type of risk it could face.

encryption of data at rest is missed).			Get every team member involved too thinking through the last safety rules (check) (SQUARE Step 9) to catch inconsistencies or omissions. Fix each weakness by building in safety rules or protective features right from the start. If you're unsure, write down the risk - suggest a way to reduce it or leave a tip for later fixes.
Malware simulation escaping the sandbox environment and affecting the host or network (in an implementation scenario, this would be catastrophic – e.g., a ransomware encrypts not just the simulated disk but the host system).	Low	High	Build it so things stay separated - use trusted virtual setups or containers to keep tests locked down. We'll make sure attack trials happen only inside cut-off virtual machines, blocked from touching the host's files or reaching outside networks. On top of that, skip real malware code when building the model; go with harmless stand-ins that mimic malicious behavior instead. Once it's being built later on, hammer the setup with security checks to confirm nothing leaks out. If

			something odd pops up - say, a strange process showing up on the main machine - the sim shuts off right away as a backup move.
Time management and integration risk (delays in one phase affecting others – e.g., diagrams taking too long, leaving insufficient time for threat modeling and documentation).	Medium	Medium	Follow the project schedule like a roadmap - keep eyes on it tight (Muneera's call as lead). When any stage begins lagging, pull together a quick huddle to shift duties around or run steps side by side. Say design visuals fall behind - Aljazi could kick off threat analysis using rough sketches instead of holding back for polished versions. We've slipped in a seven-day cushion pre-handover for merging pieces, just enough to handle small holdups. Weekly check-ins keep big shocks from popping up at the end.

Here's how we looked at risks: instead of numbers, we picked simple terms like low, medium, or high to rate how likely something is to happen and how bad it'd be. Take "low chance" - that just means right now, the team doesn't expect it'll occur. On the flip side, a "high effect" warning says if it does go down, things get rough - could mess up the project big time or break security. There was one case - the sandbox leaking malware - that seemed super rare, so almost zero odds, yet the fallout would've been massive. To keep things clean, we labeled that one low/high even

though technically it's more extreme; still, we watch it closely 'cause you can't afford to slip there.

This risk strategy grows with the project. At every stage - particularly when checking progress under the Spiral approach - we looked again to spot fresh threats or shifts in odds and effects. Instead of waiting, we tackled problems early, which boosted our shot at succeeding while building a safer system.

Functional and Non-Functional Requirements

This part shows the functional needs (FR) - what the system should do - and also covers the non-functional ones (NFR), meaning how well it must perform or what limits apply. Every need fits the SMART rule - clear, checkable, realistic, useful, and verifiable - while matching the project's aims and known issues.

Functional Requirements:

FR1: User Authentication and Roles

The system will ask people to sign in with a name plus a secret code. It works with no less than two kinds of access levels: Learner (standard person) or Admin. After signing in, what you can do depends on your level - like how only admins get into control tools.

Measurable: When users reach the login screen, it shows a request for their details; entry happens solely after accurate info plus an approved role tag. Once signed in, the system reveals which role they have along with related features available.

FR2: Scenario Selection

A trainee who's signed in can look through different cyberattack situations and pick one they want to try. While checking them out, each situation comes with a straightforward explanation along with settings that you can tweak - like launching a DDoS strike against a website where strength levels change, or setting off ransomware on a single computer to see how it spreads.

Clear goal: Two set situations - like DDoS or ransomware - show up in the menu. Once picked, the person moves on to setting things up right away.

FR3: Defense Configuration

When someone runs a test situation, they should first tweak how protection works. They might turn a firewall on or off, pick rules for spotting threats - like catching every virus or using basic detection - or adjust caps on system usage. Without any custom choices, it simply uses built-in safety levels.

A clear target: on the scenario setup page, there should be no fewer than two adjustable security options - like a toggle for firewall or a menu to set how alert the intrusion detection system is. When these protections are switched on or off, the way things play out in the sim needs to shift accordingly - one result with them active, another when they're not.

FR4: Launch Simulation

The user can kick off their selected cyber attack sim with just one click - like hitting a "Run Sim" button. Once started, the setup builds itself, pulling together virtual networks and devices on its own. Then it plays out the attack plan without needing more input.

A measurable outcome means once the sim starts, things kick off right away - with clear signs like live updates or running logs popping up. As the assault plays out fast, almost instantly, users get to watch each step go down, no delays.

FR5: Monitoring and Feedback During Simulation

While a sim runs, the system gives constant updates right away - like live visuals of network flow when hit by DDoS, warnings from intrusion tools spotting breaches, or time-stamped logs tracking how far an attack's gotten. With the DDoS case, users get a real-time chart showing how many packets are coming in and where the firewall's blocking them.

When it comes to ransomware, the system shows activity like when files start getting locked up or threats pop up. As long as people can watch what's happening while things run - not just at the end - this part checks out.

FR6: Simulation Result Reporting

When the sim finishes - or someone halts it - the tool fires off a full write-up without delay. That breakdown covers major moments: the kind of assault launched, which safeguards kicked in, what damage showed up, along with whether protection held or collapsed.

Plus, it packs useful numbers like intercepted data chunks or locked files, tossed in alongside short tips on next steps.

A **clear label** showing which test was run - along with whether the breach worked or got blocked - needs to show up on the outcome page. Instead of just listing details, group them so it's easy to scan: what protections were live, plus any warnings that fired during the attempt. Toss a time-stamped record into storage too, so someone can go back later if they need to check what happened.

FR7: Administrator Scenario Management

Admins get into a dashboard that lets them build, tweak, or wipe out attack setups while adjusting how the system behaves. To make fresh situations, they lay out network layouts, drop in attack code, along with defense options on hand. Plus, they handle a locked-down collection holding known attack methods and virus examples.

Measurable: The admin area needs form fields to build or update situations - like title, info, and threat category. Once saved, fresh cases ought to show up in the learner's selection menu. When checking things out, we need to see one situation properly created or changed.

FR8: Administrator User Management and Oversight

Admins get full control over user profiles - making new ones, tweaking existing details, or turning them off - as well as keeping an eye on how simulations are used. They're also able to pull up records that reveal which person launched what test case, at what time, together with brief reports on the results.

A **measurable goal** means the admin panel needs tools to add, view, edit, or remove users. Besides that, it's got to show a running list - or maybe a live summary - of what simulations have run lately.

Non-Functional Requirements:

Security and Privacy:

All system info plus messages've got to stay locked down. User logins, situation details, or any report need encryption whether sitting idle or moving through networks via TLS. Access based on roles makes sure admins or trainees only get what they're allowed - no overlap.

Measurable: This setup uses HTTPS to secure online traffic, along with locked-down databases. During checks, testers need to prove private info stays hidden unless properly accessed - also making sure no one can gain extra access by exploiting flaws.

Performance:

The system will run tests smoothly while showing live visuals. It needs to handle one tough attack scenario without slowing down the display. If several people are using it, it can manage three running simulators at once.

Measurable: When tested, high traffic - like 10,000 packets each second - mustn't make the system fail. Event log responses or screen changes need to take no more than one or two seconds, otherwise performance lags; meanwhile, processor use ought stay below about 80%, assuming normal equipment setup.

Usability:

The layout's got to make sense right away, especially since lots of users using it might just be starting out or still learning. Logging in, picking a setup, hitting go, then checking outcomes - this whole path needs to feel natural without any confusion. Key moves such as launching or pausing a sim? They've gotta take no more than one or two taps to pull off.

During **casual testing**, users ought to start a session just moments after signing in - then make sense of the output on their own. The design's got tooltips that actually help, text you can read easily, plus colors that stand out well, particularly when showing warnings or errors.

Reliability and Stability:

The platform needs to deal with mistakes without crashing. When a run doesn't work - say, files are gone or the attack code is broken - it shows a straightforward warning instead of locking up or losing info. One faulty session won't mess up another since each operates on its own.

When you add test mistakes, the system must spot problems, alert the person, yet return to normal without issues. Over time, it's built to stay running more than 99% of the moment - maintenance included.

Maintainability and Extensibility:

The setup needs to break into pieces so upgrades go smooth, yet stay flexible. When fresh threats show up, plus new shields or scanners arrive, they fit right in - no touchy-feely changes to the main brain.

Achievable:

Each part needs to stand on its own - like a Scenario chunk that works apart from the core system. Notes should show how users can toss in fresh situations or tweak pieces. Settings for these cases live in files or storage spots, so changes happen without touching the program itself.

Safety (Training Environment Containment):

The setup needs to stay totally separate from actual systems. Under no circumstance can anything impact the main device or outside connections. Using virtual machines - or similar tech - keeps everything locked down tight.

During tests, fake setups link solely to private digital networks - never touching actual hardware ports. Later checks on software design need to confirm data moves just one way, blocking pretend threats from jumping into operational gear.

All listed needs fit the project's goals while tackling problems we spotted earlier. One ties to another - like testing logins for FR1 or checking simulations across various defense setups in FR3. Hitting these targets leads to a working system that does what it should, staying solid on safety, ease of use, speed, and consistency.

Security Requirements (Using the SQUARE Methodology)

To get a clear picture of what's needed for security, we used the SQUARE approach (Talreja, 2025). This method breaks things down into nine steps - guiding how risks are spotted, grouped, or ranked in an organized flow. Here's a look at every phase carried out step by step, followed by the end list of key protections sorted by importance.

Step 1: we nailed down what words mean - so no one's guessing. Our crew and the main contact (basically, our teacher playing client) settled on clear meanings for stuff like "asset" (think VMs and stored info here), yet also talked through things like "danger point" (say, hackers or nasty software doing damage). Instead of skipping ahead, we spelled out gaps in setup that bad actors might use, while sorting how likely plus harmful those issues could get.

Not only that, but unique bits tied to our work got explained too - terms such as cyber lab space, locked-down zone, flood attack types, lock-up viruses, digital gatekeepers, alarm detectors within our model's world. A shared list helped us stay synced when talking about protections or risks after. As an example, we made it plain: isolation equals zero network links and split processes from test area to actual machine.

Step 2: Figure out what needs protecting – After that, we listed the main aims for keeping things secure. Those targets linked up with broader plans while focusing on key parts like secrecy, accuracy, and access - alongside concerns such as protection from harm. What we came up with looks like this:

- **G1:** Keep private info under lock – stuff like login details, example data shown in cases, and activity records shouldn't end up seen by anyone who isn't allowed.
- **G2:** Stop changes to system setup or test runs – normal people or bad scripts shouldn't mess with how things are configured; also, trial outcomes need to reflect real events (say a firewall stopped an assault - it better have really happened).
- **G3:** Block leaks from virtual zones – harmful actions or viruses can't break free from their testing box, and whatever happens inside must never hurt systems outside. That's a big deal specific to our situation.
- **G4:** Keep training tools working at all times – even during intense simulated attacks, the service should stay online without crashing. A test like a fake DDoS shouldn't bring down the app's interface or block others from using it.
- **G5:** Make actions trackable - Who did what inside the system needs to be clear, such as who started a drill and how it turned out. That way, people can't hide misuse, since every move is logged; plus, it supports improvement by letting users study past runs.

- **G6:** Follow rules loosely - Even without strict laws applying here, we aimed to stick to common standards and any fitting ethics or legal norms - like avoiding actual personal data during tests, along with honoring license terms for tools built into the setup.

Those aims gave us something to aim for when setting needs. Like, based on G1, it's clear we have to sort out how data is locked down and who gets in; considering G4, we've got to handle system resources smartly - otherwise the platform might overload itself - and stuff like that.

Step3: build stuff that shows how things work. For this bit, we pulled together materials to get a clearer picture of the whole setup. Some pieces were ready from before - like diagrams showing actions such as "User starts attack drill," plus a rough layout of the system structure. On top of that, we spelled out risky situations where bad behavior could happen. One example? Someone sneaking in pretending to be an admin, aiming to mess with test setups.

Another one involved harmful code inside a simulated run trying to jump onto the real machine it's running on. To dig deeper into that last issue, we mapped out possible paths for the malicious software to break free - say through network links, shared storage spots, or flaws in virtual layers. These examples and visuals helped us spot weak points later on by revealing likely threats and which parts of the system mattered most.

We also recorded what parts make up the system - things like the collection of test cases, the users' data store, the simulated space, plus how solid the main machine stays. On top of that, we sketched out early guesses about setup conditions - one being that the hypervisor works right and runs current software, because if it doesn't, everything changes.

Step 4: Check for Risks – With help from earlier documents and imagined attack situations, we ran a step-by-step review. Instead of guessing, we looked at possible dangers to the setup - most tied back to ideas from our STRIDE session. Where one danger popped up, we weighed how likely it'd occur alongside its effect, sort of like the partial chart seen in Part 9 (even though that part also covers timeline hiccups).

Take the case where someone sneaks into an admin account - we rated that chance as so-so (tricking staff might work in practice), but consequences? Severe (whole system falls under outsider control). A different danger - malware breaking out through network tricks - got a look; chances are slim if the setup's tight, but the fallout could be serious. Because it might lead to actual harm, we judged how risky each case was, then checked what safeguards were already there or what gaps needed filling. From that review, it became clear where stricter rules had to go.

Top concerns popped up around fake identities gaining control, weak isolation letting things spread, and systems getting flooded so they crash. Less urgent, yet still notable, issues involved private info slipping out through exposed records or stolen data, along with blame games when proof trails went missing. The result here? A list of threats sorted by risk level - this straight-up shaped how we ranked needs later in Step 8. Take containment failing - it could cause massive damage, so rules around sealing things off shot right to the top.

Step 5: Pick Ways to Gather Info – Since we're a small group working with people inside the org and already have useful course materials, we went with two main approaches - brainstorming together and going over written resources. Instead of just talking loosely, we set aside time as a team to walk through how the system would be used - and misused - jotting down every safety-related need that popped up. On top of that, we looked at outside sources; for instance, OWASP's web security tips helped us remember things like login timeouts or rules around passwords, while NIST docs pushed us to think about tracking actions and keeping logs.

At one point, we thought about using checklists - we ended up pulling one from a textbook on security needs to double-check nothing slipped through (example: "Is there a log-in verification rule?" Yep, got it). Is there a need for checking inputs? Not clearly so far - so we should toss that in. Our "people involved" weren't many - we basically only had us, no actual users to talk to - so chatting with folks or handing out forms didn't really work here.

With just a few of us on board and everyone kinda on the same page, shooting ideas back and forth or ticking boxes off a list saved time. On top of that, we quietly leaned on misuse scenarios - as little notes - to shape what was needed; like asking “how do we stop someone from messing this up,” which then turned into a concrete must-have.

Step 6: Elicit Security Requirements – Using the chosen techniques, we developed an initial list of candidate security requirements. We captured every security-related need we could think of, even if some overlapped with non-functional requirements or seemed obvious (we would refine later).

The initial list (unedited) included requirements like: - “The system shall enforce a password complexity policy for all user accounts.” - “The platform shall only allow authenticated and authorized users to initiate simulations or change configurations.” - “There shall be an isolation mechanism (like VMs or containers) to separate the simulation environment from the host OS.” - “Network traffic from the simulation environment to external networks shall be blocked or restricted.” - “The system shall maintain logs of all simulation events and user actions, with timestamps and user IDs.” - “All logs and sensitive data shall be protected from unauthorized access (read/write) – likely via encryption and access controls.” - “Regular users (Trainees) shall not have access to modify scenarios or system settings.” - “If any simulation process exceeds predefined resource limits (CPU, memory, runtime), the system shall terminate that process (fail-safe to prevent DoS).” - “The system shall alert the Administrator in case of any critical security events (e.g., multiple failed login attempts, or a simulation triggering a containment breach alert).” - “The design shall be compatible with adding multi-factor authentication for admin logins as an extra security measure (not necessarily implemented now, but not prevented by design).” - “Input validation: the system shall validate all user inputs (scenario parameters, etc.) to prevent injection attacks or out-of-range values that could cause crashes.” - ...and more. We ended up with a robust list of ~15 security requirements at this stage (some technical, some procedural). We weren’t focused on sorting or cutting repeats just yet - just wanted every piece out in the open. From there, things got looked at more closely down the line.

Step 7: Sort Needs – We grouped the gathered needs so they’d be easier to handle while making sure nothing got missed. The groups came from ideas tied to STRIDE plus how the system works. User Check-In Stuff covered logins, rules for passwords, proving who you are.

Who Can Do What handled roles, rights, actions allowed inside the app. Keeping Data & Systems Honest focused on guarding setup files and scenario details from being messed with - also included checking inputs. Privacy & Locking Down Info dealt with scrambling data, safe message transfer, keeping personal info private. Boxed-Off Spaces took care of running sims in separated zones, like virtual machines or locked networks, limiting resources too - it was our own twist. Watch & Trace Actions involved tracking events, watching activity live. Keep Running Even Under Stress aimed at surviving attacks that try to crash it, staying up when things go bad - sometimes touched boxed-off spaces or limits again.

Each need landed in just one group even if it kinda fit elsewhere - we stuck to its main home. Like “stronger password rules” ended up under Check-In; “run sim in VM” sat in Boxed-Off; “kill task using too much memory” lived under Keep Running.”. This split showed some missing pieces - we saw lots of stuff on isolation and access limits, yet almost nothing clear around spotting issues, like catching insider abuse. Because of that insight, we tossed in a rule to notify admins when odd activity pops up, just like before.

On top of that, we spotted no direct call for encryption (sure, there was shielding logs from outsiders, but now we spelled out using encrypted storage). In short, sorting things this way acted like a quick check for holes, making sure everything got covered.

Step 8: Sort Out What Matters Most – After listing the security needs, we ranked them one by one - real projects can’t do it all at once, so choices have to be made. Instead of tackling every item equally, we focused on which ones hit the biggest risks first, how hard they’d be to build, and who needed them most. A basic ranking system was used: High, Medium, or Low. High meant non-negotiables - these fix serious dangers or keep core protections locked down.

Things like user login checks, separating systems from each other, and limiting admin powers got top rank, simply because skipping them would leave things wide open. Mid-level items still mattered a lot, yet some could wait a bit if time or resources ran short. These usually tied into layered defenses or mid-level risks. Take stored data encryption - it landed as Medium because while it matters for privacy, if passwords are hashed and little sensitive info exists, skipping full encryption briefly in early builds isn't catastrophic - just risky over time. Instead of immediate threats like broken isolation, this slips under the radar. Monitoring and tracking started out rated Medium too - they help trace actions and assign responsibility, yet the system runs barebones without them; still, real-world operations demand these sooner or later. Low Priority meant optional extras or long-term upgrades, things that sounded good but weren't urgent.

Not many items ended up here since nearly every security need seemed essential back then. Still, we rated stuff like "add MFA for admins" as Low right now - not since it's irrelevant, but 'cause it likely won't work in the initial build, plus the danger of leaked admin passwords was already reduced thanks to solid login codes and few people having admin access; adding multi-factor could come later.

At the same time, we looked at what relies on what: certain needs just need to happen earlier (like how auditing logs makes no sense until logging itself exists). What we ranked matched up with the threat analysis done back in Step 4 - any item tackling serious risks jumped straight into High tier. A ranking chart (shown below) helped tag every demand clearly.

In the end, it spelled out exactly which items were must-haves (the crew made sure these top-priority features fit into the plan), while others stayed optional if there's space or pushed off till next rounds.

Step 9: Checking Needs – Last thing we tackled was making sure our security checklist made sense and covered everything needed. A teammate went through every item out loud while others poked holes in them or asked tough questions.

Each point had to make sense, fit real-world limits - no fantasy solutions allowed - and hit the dangers we'd spotted earlier. On top of that, we scanned for repeats or things that didn't line up. Take, for instance, two access rules we combined into a single, easier-to-grasp statement. Instead of keeping them separate, we linked similar ideas to sharpen clarity. Our approach shifted toward uniform phrasing - each now starting with "The system shall..." - so they'd feel predictable when read. Think of it like tuning instruments before a show; everything needed to match in tone. Once adjusted, the set went through a quick check-up, just to catch loose ends or gaps. That final review acted like a filter, removing fuzziness where possible. With that done, what you see in the table ahead is the cleaned-up version.

Based on Table2, the last safety needs are prioritized based on their importance and functionality.

Table 2: Prioritization of Security Requirements

Security requirements	priority
SR1. Strong User Logging in is required for everyone - no exceptions. A strong password rule applies here: long enough, mixed characters included. User codes stay safe through encryption techniques like hashing. Access to run tests or view info? That's locked behind login checks only. Why do this? To block fake identities from sneaking in. (<i>The Cyber Range: A Guide</i> , n.d.).	High
SR2. Role-Based Access Control: Users get permissions depending on their role. While Trainees may just start or watch simulations, admins have extra rights - like editing situations or handling accounts. Also, checking every log entry is limited to admin staff alone. Rationale: Keep access low so breaches don't spread far - minimal rights needed. (<i>The Cyber Range: A Guide</i> , n.d.).	High
SR3. Keeping environments separate or	High

Sandboxing: Every test run takes place inside a cut-off digital space - like a VM or container - that can't reach the host system or connect outward to the web. The setup keeps the simulation's network inside the system itself - outside links need to pass via restricted entry points that normally shut down incoming or outgoing data. Reason behind it: Snap down threats so they can't break free or mess up anything important (Lockdown (Cyber Range Project Team and the NICE Community Coordinating Council, Cybersecurity Skills Competitions Community of Interest, 2023, #).	
SR4. Safe messaging & locking down data: Every website connection must run on HTTPS or TLS to keep information safe while moving between devices. On top of that, crucial stored details - like login codes, setup files, and records with private or important content - need encryption; if not fully locked, they should still have tight access limits so breaking into the server doesn't mean instant exposure. Why this matters: Keep info private from spying or stolen data - blocking snooping stops leaks. Preventing taps means security stays strong through hidden transfers.	Medium
SR5. Input Validation on Configuration: Check settings carefully before saving them The system checks every input from users - like scenario settings or custom labels - to make sure it's the right kind, size, and structure, tossing out anything suspicious or beyond limits. Say someone picks how many attack packets to send - it locks that number within safe bounds fair upper cap; when text boxes come into play, block possible script hacks. Why? Stop people from abusing the system using tricky data entries - like tampering or injecting harmful code.	Medium
SR6. Keeping records of actions taken + tracking changes over time The system keeps a record of key actions - like login attempts (whether they work or	Medium

fail), starting or stopping scenarios, when defenses turn on, and any alerts or glitches. Each entry notes when it happened, who was involved, and what exactly took place. No one's allowed to alter or wipe these entries without permission; only Admins have that access. Reason: so you can trace back who did what and sort out issues if something goes wrong.	
SR7. Resource and Performance Limits: The system must set boundaries on how much power each simulation gets so others aren't disrupted - like capping CPU or RAM usage per virtual machine. When a test run goes wild with too much processing or gets stuck looping nonstop, the setup needs to spot it fast then shut down just that session. In case things are already maxed out, incoming jobs either wait their turn or get turned away without delay. Rationale: Prevent platform crashing from misuse or too many users at once (Accessibility) (<i>Spiral Model: The Risk-Minimizing Model for Software Projects</i>, n.d.).	High
SR8. Warnings about safety plus Watching: The tool needs ongoing checks to spot key safety issues. When something major happens - say, lots of wrong logins hinting at a forced break-in, or a drill sets off a sign that someone's trying to sneak past barriers - an alarm pops up for admins, maybe on screen or by message. Why? Spotting trouble fast detecting threats against the system plus fast action by admins (spotting break-ins on the platform).	Low
SR9. Configuration Integrity & Changes to setup details need approval plus tracking. For official cases, the system checks file safety - say through checksums or digital tags - to block meddling from bad actors. When upgrading the platform, updates roll out without breaking current protections; this means update tools are built tight from the start. Reason: Stop people from changing the system's stuff without permission - or messing	Medium

with it on purpose - so things stay safe and work right.	
--	--

(Priorities: High = must-have, Medium = should-have, Low = nice to have or later plans. Each need's category in reasoning links to STRIDE when it fits.

All high-priority needs - SR1, SR2, SR3, SR7 - match up with core protections the system can't do without: verifying who's using it, restricting what they're allowed to do, keeping risky tasks separated, also making certain internal tests don't crash the whole setup. Mid-level items include helpful layers such as data scrambling or tracking activity, whereas lower-ranked ones add extras - for instance warnings sent to admins - that boost resilience but aren't essential right away.

These security needs were matched up against our threat analysis, making sure every key risk has a specific countermeasure. Take user impersonation - SR1 along with SR2 handles that; malware breaking out? That's covered by SR3; denial-of-service attacks on the platform fall under SR7's scope; unexpected alterations get blocked thanks to SR2 or SR9 stepping in; missing audit trails are fixed through SR6 - and similar links exist across others too. So, using the SQUARE method led us straight to a full list of protections, shaping how we built the system securely from the start.

System and Design Modeling

- **High-Level System Architecture**

Overall, this system runs on separate layers - one handles how users interact, another manages main functions, while the third deals with simulated scenarios - all kept apart deliberately.

The User Interface Layer works as the front part - a browser-accessible panel where learners and managers sign in, pick situations, or check outcomes. It connects safely through HTTPS to the app server.

The Application Logic Layer - also called the Simulation Orchestrator - acts as the main hub. While it handles login tasks through the Session Manager, it pulls ready-made attack setups via the Scenario Manager. As the Environment Manager sets up virtual spaces, control shifts between offensive moves and protective actions using Attack Controller & Monitor. Throughout every run, live stats flow into the Data Logger from ongoing activity. Once things wrap up, results get broken down by the Reporting Module so admins and users can make sense of what happened.

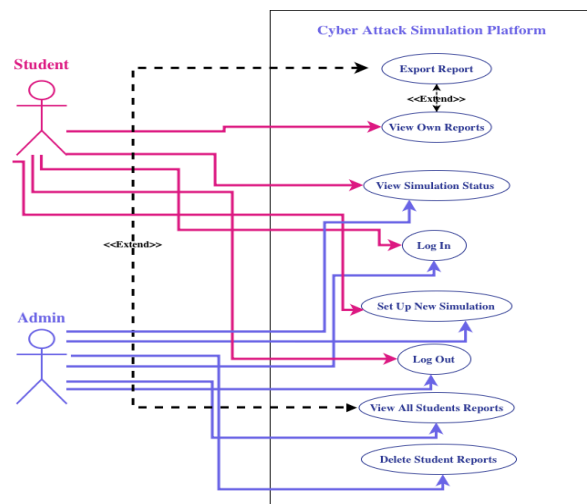
The Simulation Environment Layer's a cut-off digital zone where offense meets resistance. Every setup fires up short-term VMs - ones for hackers, ones for victims, ones for guards - that talk through a locked-down net.

Beyond that, data storage keeps user accounts, setup details, plus records - each piece reachable solely via approved app parts.

Lastly, the setup keeps things well split - interface, brainwork, and fake battlegrounds don't mix, so tests stay safe while spreading easily over various machines.

- **Use Case Diagram**

Figure 1 Use case diagram:



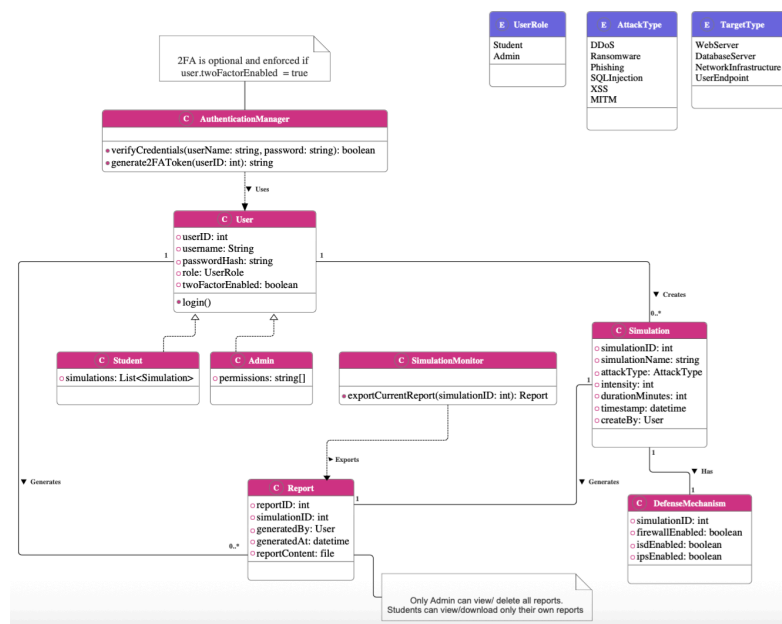
This figure 1 explains how two key users - Student and Admin - work with the Cyber Attack Simulation Platform. While one logs in, the other might review ongoing simulations. Creating a new simulation is possible for both, though they may do it at different times. Instead of sharing tasks exactly, each exports reports when needed. After finishing their work, either can sign out whenever ready.

Young learners can check out or download just their personal results. On top of that, staff members get broader access - they're able to look through every pupil's file and wipe reports when it's necessary.

The 'extend' connections from viewing to exporting reports suggest export follows only once someone's looked at the report. In general, the chart highlights how each role gets different levels of access and what they're allowed to do.

- **Class Diagram**

Figure 2 Class diagram :

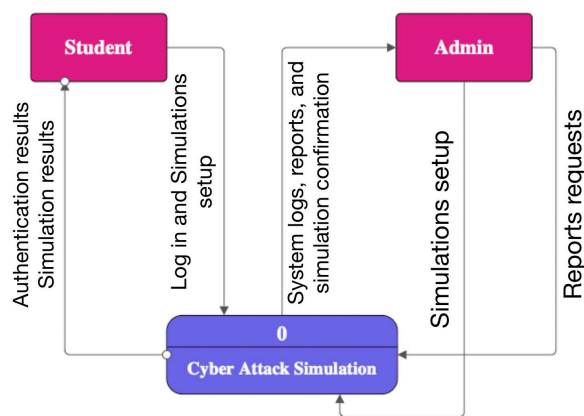


This figure 2 shows how the Cyber Attack Simulation Platform is built. In the middle sits the User class - this gets split into Student and Admin, where their actions depend on what's set in

the UserRole list. Each one logs in using the AuthenticationManager, a system that might use second-step checks if turned on. People running the platform can start up Simulations, adjusting things like attack kinds, targets, strength levels, or how long they run. Every run ties into a defense setup - like a firewall or IDS - and gets watched by a SimulationMonitor that lets you export results. Reports pop up for each test, but who sees what's limited: learners check only their own, admins peek and wipe every one. The AttackType and TargetType lists keep data entry neat and tidy. This layout shows how pieces fit together cleanly, security baked in, rules about who does what spelled out tight.

- **Data Flow Diagrams (DFD)**

Figure 3 Data Flow Diagram level 0 :



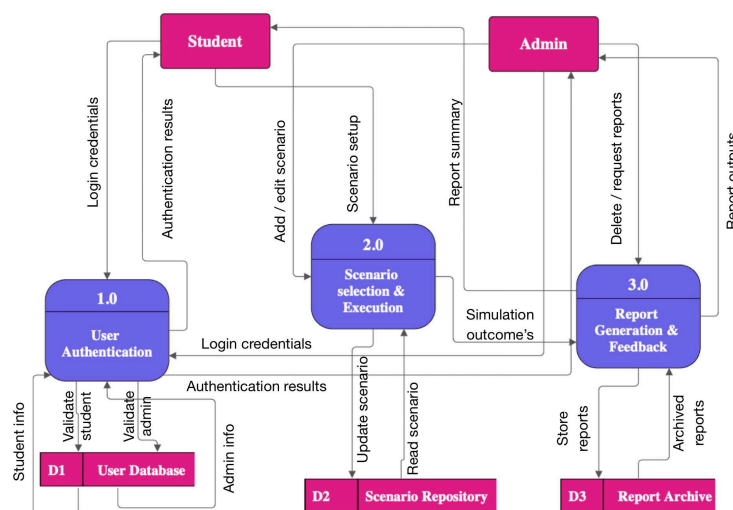
This figure 3 shows the cyber attack simulator as one box. From the outside, the Student - just like the Admin - feeds data into it. Data comes back out their way, depending on what was sent in. Details move both ways, hopping across clean lines that link pieces together.

Some learners give away their sign-in details before requesting a test scenario and looking at what happens next. Meanwhile, staff carry out these tests while watching who's doing what and which reports get opened. As things kick off, the platform collects every piece coming in,

processes each part one after another, then fires responses - like outcome breakdowns or go-ahead messages returned straightaway.

A straightforward view of how data moves from users to the system - without diving into what happens behind the scenes. As it follows input leaving the person, internal operations remain untouched. Rather than explain computations, it focuses on where things travel. When you're pushing or pulling info, just the outside journey counts. Even if core mechanics aren't shown, who sent it and which way it goes is easy to see.

Figure 4 Data Flow Diagram level 1 :



This figure 4 shows the cyber attack simulator as one box. From the outside, the Student plus the Admin feed data into it. Data comes back out to them depending on their input. Details move back and forth across each side through visible links connecting pieces.

Some learners give out their sign-in details before requesting a test run and later review what happened. Meanwhile, staff members carry out these tests while also watching over account data as well as who views reports. As things kick off, the setup collects every bit coming in, handles each part one at a time, then fires back answers - like outcome snapshots or go-ahead messages. This Level 1 Data Flow Diagram splits the Cyber Attack Simulation Platform into three main actions.

Step one handles sign-in tries, verifying details through the User Database (D1) whenever someone logs in.

Process 2.0 – Picking and Running Scenarios allows Students or Admins to build and launch practice runs, grabbing required info from the Scenario Storage (D2).

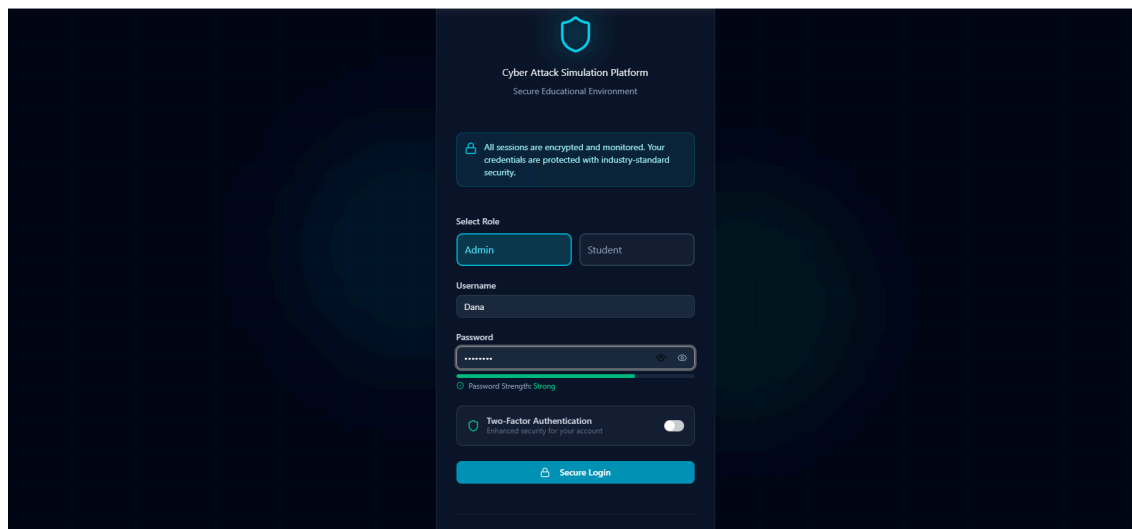
Step 3.0 – Making Reports & Getting Input manages how sim results are turned into documents, saving each one in the Report Storage (D3).

Admins get access to every report, but learners just spot comments linked to their personal sim runs. This chart shows clearly how info flows inside - from people to actions to storage spots - in the setup.

A straightforward view of how data moves from users to the system, minus what happens behind the scenes. As it follows input leaving the user, backend tasks aren't shown. Rather than explain computations, it focuses on where things travel. No matter if you're pushing or pulling data, just the outside flow counts. Even though the inside mechanics stay out of sight, where stuff starts and which way it goes is easy to see.

- **Interface Design (Figma Prototype Screens)**

Figure 5 Login with 2FA :



As it shown in figure 5, The user can choose if they're a Student or Admin, then type in their username along with a password - there's also an option to turn on two-step verification for extra safety when signing in.

Figure 6 Dashboard (simulation overview) :

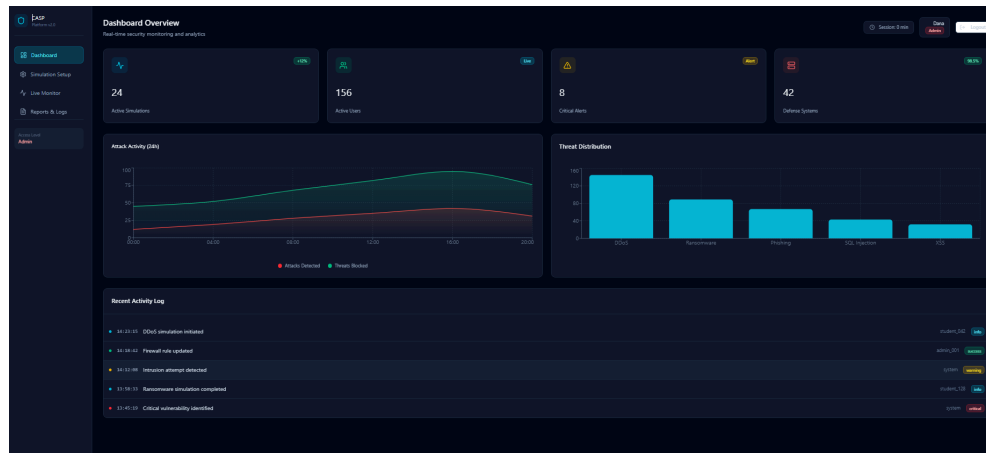
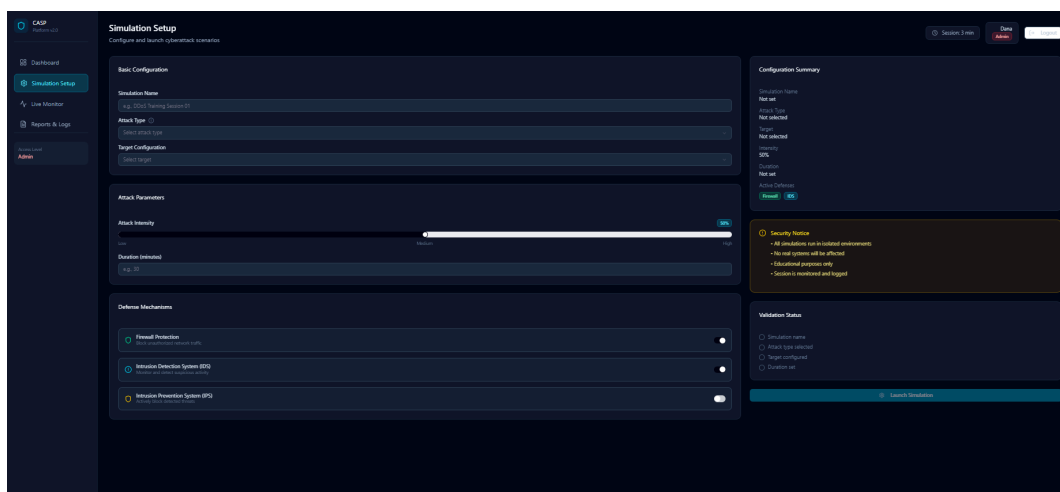


Figure 6 shows simulations statuses across the dashboard graphs - sidebar links lead straight to Simulation Setup, Live Monitor, and Reports & Logs. Each section opens fast through the side panel that holds Setup alongside live tracking and log history.

Figure 7 Scenario setup (select attack, defenses) :



In Figure 7, User sets up tests by adding a title, selecting attack type and targets, adjusting the attack intensity, and also turning on security features such as Firewall, IDS, or IPS.

Figure 8 Live monitor (traffic chart, IDS alerts) :

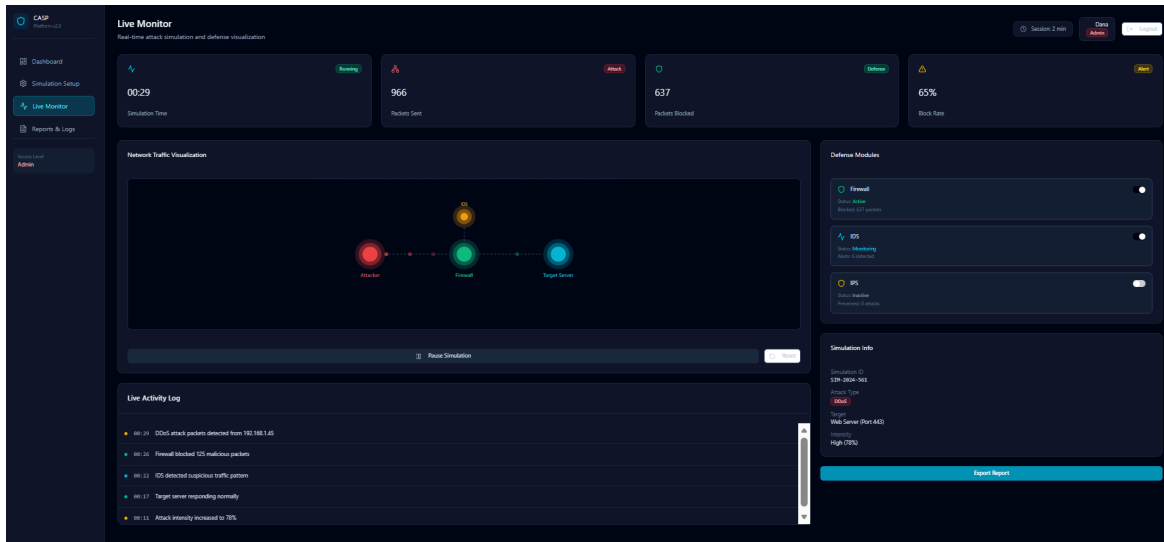


Figure 8, User can monitor what's happening live, then grab a real-time snapshot of the results using the export report feature so you can view it later.

Figure 9 Reports view and export (Admin) :

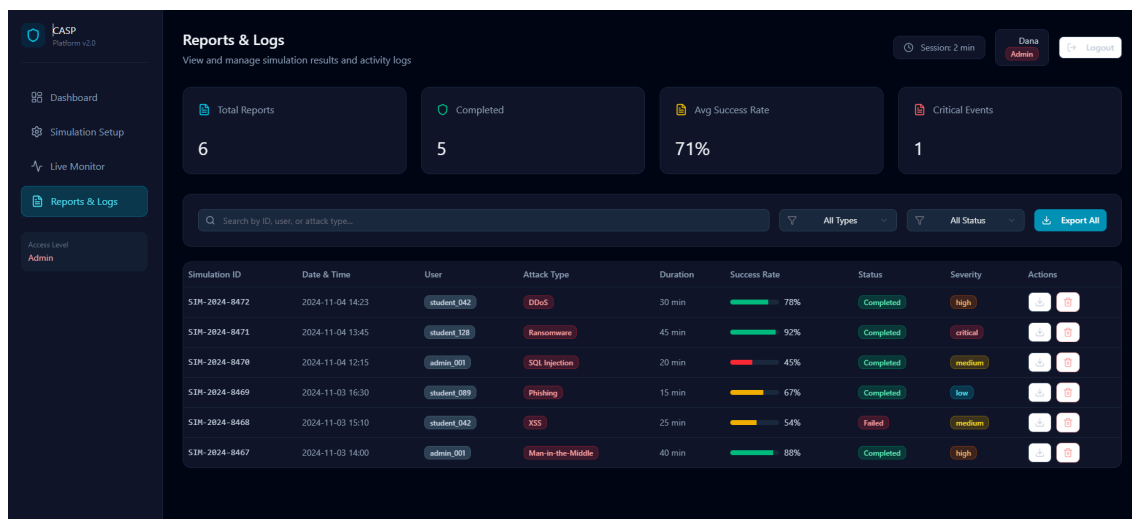
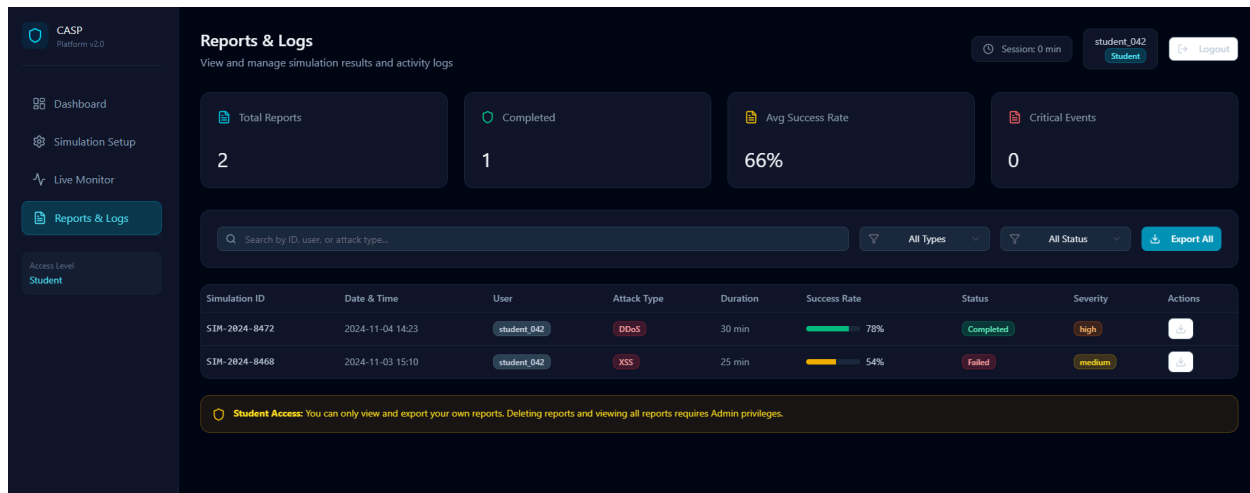


Figure 10 Reports view and export (Student) :



User can view the reports and log based on their privileges. Based on Figure 10, Students are able to check and download their personal simulation reports. Admins have access to view, download, or delete any report stored in the platform as Figure 9 illustrates.

● Secure Architecture Principles in Design

We built our system using key security ideas, making sure safety runs deep through how it's put together and works day to day. One idea backs up the next, stacking safeguards that cut down dangers coming from inside or outside the network.

Principle of Least Privilege

We followed the Principle of Least Privilege (PoLP) across all layers. Every person, task, or part of the system gets just enough access to do what it needs - no more. Take trainees: they're able to launch practice runs and check personal outcomes, but editing situations or peeking into others' info isn't allowed. Those powers sit with admins, who handle setup changes along with account control.

Behind the scenes, sim tools have tight limits too - they're blocked from changing configs or touching user profiles; only the main control program handles those tasks.

Inside sandboxed spaces, malicious scripts still run without elevated rights, so their damage potential stays low. Much like that, access to databases gets locked down: the test coordinator pulls info but can't modify or wipe structures. That rule means when something breaks through, the invader can't spread far.

Defense in Depth

The setup uses layered defenses - so even when one safeguard breaks down, backup measures stay active. On the network side, simulated environments run separately thanks to virtual setups. Inside each app, login controls along with tight data checking block intruders. Even if someone slipped past front-end rules, the backend would double-check inputs prior to any action.

All key moves get recorded, so you can trace odd activities later. In the test zone, safety builds up step by step - one example: a firewall stops unwanted access whereas an intrusion detector keeps watch. Suppose someone slipped past those, they'd run into locked-down virtual machines, system-level limits, along with further roadblocks. Each task - like setting up a manager profile or adjusting a setup - goes through several checks back to back, making sneaky actions way less likely.

Secure Defaults

The system starts with safety settings already turned on. If someone runs a test situation, protective tools like firewalls or intrusion detection kick in by themselves - unless that person chooses to switch them off. Rules for passwords make strong formats mandatory, while fresh profiles begin as standard users instead of high-level admins.

Usually, the test area stays offline - this keeps harmful code from leaking out. When logs stick around, they're wiped after some time so private details don't stay stored. If odd stuff pops up - say, a weird scenario ID - the setup freezes instead of pushing forward blindly.

Isolation and Sandboxing

One big focus? Keeping things apart. Every test works inside its own little world - built using individual VMs or containers. No sharing between these setups, none with live systems either. That way, when different people run tests together, nothing crosses over.

The setup keeps each part separate by design. Instead of hitting the database straight, the frontend talks via locked-down backend routes. Same deal on the back end - tasks like login checks, sim rules, or management tools run apart so one glitch won't crash others. Sensitive records and activity trails stay off-limits to everyday users, cutting leak risks.

Complete Mediation

The idea behind Complete Mediation is that each time someone tries to get in, their permission gets confirmed first. Every move a user makes - such as launching a sim - goes via the Simulation Orchestrator, which double-checks active logins, assigned roles, and correct scenario codes. Hitting internal tools or data straight isn't allowed; everything flows through one main checkpoint instead. That way, hackers can't skip safeguards or fake high-level orders.

Open Design and Transparency

Our setup doesn't hide flaws behind secrecy. Rather, it leans on clear, tested safeguards you can check yourself. For data moving across networks, we apply common encryption tools. On top of that, secure container platforms - say, Docker or VirtualBox - keep things separated and contained. Even if outsiders studied every part, they couldn't break in easier. That's because real safety lies in solid locks, separation layers, plus tight access rules. No sneaky entry points exist here. All steps follow open guidelines anyone can verify.

Separation of Duties

We split up responsibilities so no one person or action controls everything. Since admins handle settings and permissions, trainees just launch practice runs. For bigger teams, you might bring in coaches or reviewers later - yet even now, dividing duties cuts down on mistakes or misuse. On top of that, the code keeps things apart - separate chunks deal with admin jobs, user actions, and sim mechanics.

Fault Tolerance and Fail-Safe Design

The setup's designed so that when things go wrong, they do so safely. When one piece acts up, everything shifts into a secure mode right away. Say the simulation control crashes halfway - then cleanup kicks in by itself, stopping stray tasks from lingering. Should login tokens run out, you're pushed back to sign in fresh rather than stay connected on assumption. Bad data gets shot down immediately; meanwhile, if demand spikes past normal bounds - for instance, under assault - the sim shuts down flat to keep services online.

Auditing and Accountability

Auditing's baked right into how the system works. Each major move - like signing in, kicking off a sim run, or tweaking admin settings - gets stamped with time and who did it. That way, when something goes sideways, you can track back through events without guessing. It doesn't just help during probes - it also keeps people honest because they're aware their steps aren't invisible.

Threat Modeling and Security Analysis

We ran a threat assessment with the STRIDE approach - a method that checks risks across six areas: pretending to be someone else, altering data, denying actions happened, leaking info, shutting down services, or gaining extra access. In every area, we spotted possible dangers to our system, looked at what could go wrong, then figured out ways to reduce harm, all included in Table3.

Table 3: Threat modeling and security analysis table

Threat (STRIDE Category)	Description and Potential Impact	Mitigation Strategies - Strong Authentication:
spoofing	A hacker might try faking someone's identity - like signing in as a different person or an administrator - by grabbing passwords or active session keys. When it works, access gets unlocked hacking into systems - big problem if it's an admin login being faked. impact :Info gets leaked, plus changes made without permission, like messing with active setups.	Set up solid password rules while using hash plus salt to keep logins safe if hacked DB won't yield passwords easily). Possibly add 2FA to admin accounts - this boosts security. or - Use session Keep sessions safe by setting short lifespans on access keys, while linking each one to how someone logs in - like their gadget ID or network trail - to slow down impersonators. Tokens stop working when you log out or go idle too long. • Watchdog system: When a boss signs in from some fresh spot - or there's a string of bad tries - ping the actual admin right away, just in case someone's faking their way in (Guest User, 2023)
Tampering	A hacker or bad software may try messing with data or system parts -	- Access Rules & Checks: Ordinary users can't modify key data spots

	like changing scenario files to sneak in dangerous scripts or adjust challenge levels, shifting logs to hide actions, or infected simulations tweaking the core control memory. impact: Training trust drops (someone cheats by making tasks easier), plus risks to safety if settings get switched off.	like scenario storage or log files - just admin tools handle changes, yet even then every input gets screened first. To confirm file safety, the app checks things like digital fingerprints or keeps copies locked down in protected databases(Al-Fayez, n.d.). - Folder Rights Setup: Config files live outside public reach, tucked into hidden folders where only the running program or authorized staff hold editing rights. - Code Marks & Fingerprints: When dealing with crucial scripts or malicious code examples, an administrator might attach encrypted tags. During execution, the system double-checks these marks to spot unauthorized tweaks. - Locked Virtual Setups: Test virtual machines start up from fixed blueprints nobody can change - if harmful software tries messing with base systems or templates, built-in isolation layers and core OS shields block it instantly.
Repudiation	A person might do something - say, launch a test or tweak a config - yet	Every key action - logging in, starting or stopping simulations,

	refuse having done it afterward; meanwhile, someone inside with bad intent could wipe activity records to hide what happened. Take a new hire abusing system access, then saying their login got stolen; conversely, a tech lead might alter critical data and simply disown the move. Impact :It becomes nearly impossible to trace who did what, so sorting out issues slows down - and rule-breaking goes unchecked.	changing settings - is recorded with exact times and who did it. These records go into files that show if anyone tries to mess with them, like using write-once storage or sending copies offsite securely. Normal users aren't allowed to see or change these logs at all. Admins can check them, but wiping entries might be blocked - instead, they could flag old ones as saved rather than erase anything fully. Another option? Ship log backups now and then to a locked-down spot where even admin powers don't reach easily. Before removing big things - a user account or test setup - the system may ask "Are you sure?" or force a green light from another manager when risk is high. This leaves proof the move was meant. - Ways to block denial: Use e-signatures on key actions now and then. Say, log an admin shift with a verified mark from their login info, making it harder to back out.
Information Disclosure	A hacker might sneak a peek at login details or grab simulation	When moving through networks, data's protected using HTTPS plus

	outputs by listening in. One person could accidentally view someone else's run data too. Logs may hold private stuff - say, what a ransomware file does or system specifics - and those might slip out. This kind of leak risks passwords, personal records if kept, or secret methods like custom attack models.	TLS - this keeps login details or active sessions safe from snooping (Talreja, 2025). Cookies and tokens online get flagged as secure while also being set to HttpOnly. Data stored on disks? It's locked up too - either the whole database is scrambled, or just key parts like passwords, personal info, or secret project bits. Who gets in? Only those who should; each person sees only what belongs to them. Say you run a sim - the output sticks to your ID, so no one else pulls it up. Try typing someone else's report link? Blocked. System checks every time before showing anything. Logs stay clean - nothing juicy goes in. No full passwords dumped there, no private keys slipped into records. Logs usually keep track of actions and system details instead of saving what users type unless absolutely needed. – Check security now and then by scanning for typical flaws - say, making sure folders aren't publicly viewable or APIs don't accidentally spill data - so you spot leaks before
--	--	---

		they become problems.
Denial of Service (Dos)	A hacker may attempt to flood the system until it goes offline. That can occur through misuse of the simulator - say, someone intentionally starts several intense simulations at once, eating up memory - or from external traffic flooding the website interface. Another way is a badly set-up exercise that drains power, like an attack code stuck in repetition, maxing out processor use. Result? Real users get locked out of practice sessions, messing with progress or daily tasks. The machine might even shut down unexpectedly or, in extreme cases, wipe stored info if no safeguards kick in.	There's a cap on how many sims can go live at the same time - any extra ones get held back or turned away. Instead of allowing full access, each virtual machine or container used for a sim is locked to specific CPU and memory levels right from the host layer, meaning no runaway process breaks past set boundaries (<i>What Is Spiral Model in Software Engineering?</i>, 2025). To stop one person hogging resources, individual users are limited - say just one active sim per account, plus maybe a daily allowance. When a sim runs too long, like more than five minutes, it gets shut down automatically - that keeps things from dragging on forever. Before any test setup kicks off, inputs are checked; if someone tries loading a wild number like "a million fake attackers," the system swaps it out with a safer top-end value (Al-Fayez, n.d.). In case outsiders flood the site trying to crash it, defenses include shielding the whole app through firewalls or

		cloud tools built to handle traffic spikes. Scale things up - or just make sure your server's strong enough. If part of the system gets swamped, it could drop less vital tasks - say, halting fresh simulations - for a bit, so key stuff keeps working, like sign-ins or active runs. Admin access? That stays on fast track, letting staff step in when needed.
Elevation of Privilege	A hacker or shady operator might exploit weaknesses to grab more control than intended. Take, for example, someone logged in normally stumbling on a bug that hands over administrator access, or harmful software inside a virtual machine sneaking out to run commands on the main system with boosted powers. Consequences? A win here means total takeover - unauthorized actions like wiping files or planting hidden entry points become possible, while rogue programs may spill beyond their container, hitting other accounts or even the underlying hardware.	Every time someone tries a special action, their role gets checked again – no exceptions. Hidden admin links? Still need a proper token; just finding the address or switching roles on your end does nothing (Talreja, 2025). Client-side rules aren't trusted alone here. Instead of relying on code shortcuts, we make sure secrets aren't baked into the app and remove any backdoor-like features. That means zero sneaky switches that grant full access by accident. On servers and operating systems, even background tasks run with minimal permissions – only what they absolutely need. The web

		app operates using a limited system user - no root access, so breaching it won't give full control. Meanwhile, the virtualization layer works strictly with minimal required permissions. Network barriers on the main OS block VMs from touching administrative tools. So when a virtual machine attempts escape - say, exploiting a bug - it hits several roadblocks before going further. – Update consistently: Refresh all parts of the stack regularly, such as the core monitor, base operating system, and external code packages, eliminating common flaws attackers use to gain higher access (for instance, breaking out of isolated environments through documented bugs). – Test actively & inspect code: Run frequent hands-on evaluations centered around gaining extra rights - examples include altering request values like profile IDs to view unauthorized records or submitting executable files as regular users to check whether they run with elevated power. Fix any
--	--	--

		issues right after discovery.
--	--	-------------------------------

Every risk mentioned earlier shaped our approach, while fixes were either put in place or mapped out. Using With STRIDE, our setup got checked for different kinds of attacks - some clear-cut, like outsiders forcing entry, others sneakier, such as insiders abusing access or hidden flaws in structure.

Instead of just listing fixes, we tied them back to specific rules made before - SR1 and SR2 handle fake identities, while SR3 plus SR7 deal with overload risks and unauthorized control jumps; meanwhile, SR6 tackles proof gaps when actions aren't traceable - making sure each rule fights real dangers it was meant to stop. (Talreja, 2025).

Because threat modeling keeps going, we'll update it when the system changes or new risks pop up. Even so, right now we're sure the big threats are covered by our setup and strategy. This gives us a system built to handle frequent attacks and improper use, which means it offers a safe space for running cyber attack tests.

Evaluation and Discussion

Building this secure software effort felt tough at times yet taught me a lot. Here's what came up - key compromises, hurdles faced, because of how we shaped the Cyber Attack Simulation Platform.

Trade-offs and Design Decisions:

One big challenge early on was weighing realism against keeping things safe. Going with real malware or mimicking attacks exactly could've boosted authenticity, yet that path brought higher risks and headaches. So we chose to mimic outcomes - like files locked by ransomware - with safe scripts rather than active threats. It lowered danger without losing learning impact - a choice we made on purpose to keep control.

A different balance was about how easy it is to use compared to safety. Because the system's built for learners, the design needed to feel straightforward and clear. Still, keeping things secure mattered just as much. We talked through options such as two-step logins along with rigid password rules yet realized too many hurdles might annoy people trying to get in. In the end, we went with solid login codes paired with automatic sign-outs after idle time - safe enough but not a hassle.

Speed mattered too. Running a complex model using several VMs plus constant logs might feel authentic, yet it'd drag down speed and need costly hardware. So we stuck to one server running virtual systems instead. That kept things quick and accessible, but still let attackers and defenders do real moves.

We went with the Spiral model for building things step by step. It came down to this - moving slow because of constant check-ins and reworking plans, but getting a much sturdier end result. Instead of rushing ahead, we focused on doing it right and keeping risks low, especially 'cause when safety's involved, being accurate beats finishing fast.

Turns out that choice worked well: spotting threats early using STRIDE let us catch problems such as shaky logs (that's a denial risk), so we tossed in solid logging during the following round. Had we skipped the back-and-forth nature of Spiral, patching stuff later would've taken way more effort and cash. (Talreja, 2025) (Al-Fayez, n.d.)

Challenges Faced:

The main issue? Getting the SQUARE method to work well with students. Since it's usually rigid and organized, we ran into trouble - there wasn't an actual client involved. Instead, we played different stakeholder parts on our own while inventing believable misuse scenarios from scratch. Writing everything down took longer because of that, yet it helped us really grasp how systematic security requirements are built.

Figuring out the setup for the test space came with hurdles. Since we needed to mimic DDoS attacks along with how ransomware acts, digging into those tools was a must - though keeping

things secure mattered just as much. At one point, talk shifted to whether Docker setups would work better than full virtual systems when separating processes.

Sure, containers used fewer resources; however, they also opened up more danger when dealing with threats that behave like real malware. We went with virtual machines - bulkier yet more secure - once we compared isolation versus manageability. That choice mattered technically, which pushed us to dig into how actual security systems work.

A big headache? Keeping all the diagrams and models matching. Tweak just one thing - say, tossing a "Logging" step into the Data Flow Diagram - then boom, you've got to fix the class diagrams along with the use case write-ups. Doing those updates by hand felt endless, mainly 'cause we only had simple apps like Draw.io or Lucidchart. Still, after several rounds of team checks and edits, everything finally lined up.

Figuring out what the project needed took effort too. At first, we wanted big tests - lots of attackers, smart coaching bots, worldwide risks - yet deadlines and size issues pushed us toward simpler goals. Trimming things down actually made the model idea clearer and tighter.

Lessons Learned:

The project taught us a few things we will not forget:

Security's got to be part of the plan right away. Because we mapped out risks early, it made a big difference - spotted flaws like no proper data scrambling before things went too far. That gave us time to sort issues without stress. Trying to bolt on protection after everything's set? Way tougher, way messier.

Clearly, well-organized specs make a real difference. When details were fuzzy, the design team got tripped up. After sharpening terms with SQUARE, things started flowing better. That showed us solid requirement work lifts design results - no doubt.

Doing things step by step made a difference. With the Spiral method, we kept going back to fix and refine - no need to hurry through. Every loop brought sharper focus, tighter coordination, or improved safety measures.

Teamwork made things better. Every job had weight - because Aljazi shaped needs, Dana's designs shifted based on that, while Muneera used those choices to guide her write-ups and safety checks. Frequent talks stopped people from working blind, keeping details lined up. We learned protection doesn't live in one spot; it spreads through jobs and fields.

When limits were tight, we stayed grounded. Since perfect fixes didn't work, we picked options that actually fit. Rather than building the whole setup, we ran test cases in our heads based on how it was drawn up. Those run-throughs checked if things made sense - all while staying inside boundaries.

Getting ahead of problems actually helps. When we spotted trouble fast - say, looming holdups or missing skills - we skipped bigger headaches down the road. Take step-by-step records: they kept things moving, no matter who was out sick or busy. Turns out, prepping for people hiccups feels a lot like guarding code from flaws in solid app building.

In short, the work proved that building secure software isn't just writing code. It involves organized steps, thoughtful architecture, spotting threats ahead of time, also working together consistently. Each stage brought obstacles; however, clear preparation, talking openly, In addition staying flexible helped us push through. What we learned - like prioritizing safety from the start, improving step by step, being alert to risks - will shape how we handle coming school or career tasks.

Future Work

Even though we managed to create a solid design for the Cyber Attack Simulation Platform, some upgrades could help it feel more real, useful, and safer down the line. Here's what should come next.

Prototype Implementation and Testing:

The very first thing we should do is build a basic working model. Making a mini version - just one full use case from beginning to end - lets us see how it actually works when used. That means trying out limited tests, like simulating a DDoS attack, so we can check speed, reliability, and how it holds up when pushed hard.

When the model's live, real-time checks along with security probing help confirm whether our original ideas were right. This test means running common hacks such as XSS or SQL injection through the website, while also checking if a sandbox breakout can happen from inside a virtual machine. Outcomes will shape how the setup and protection measures get improved.

Expanded Attack and Defense Scenario Library:

A major upgrade would mean testing more kinds of fake cyberattacks. Coming updates might feature scams that mimic real emails, attempts to mess with databases using code tricks, or complex chain reactions - say, starting with stolen login info then leading to locked systems demanding payment.

In the same way, tossing in extra protection tools like live monitoring software or automated flaw finders could help learners adapt faster. Every new situation needs close safety checks so nothing spills outside its zone. Growing this way helps shape the system into a full-on practice ground for digital defense skills.

Multi-User and Team-Based Capabilities:

Right now, the setup works for just one person at a time. The next logical step? Letting multiple people join - say, for team drills like Red versus Blue games. One side tries breaking in, the other blocks them. To make that happen, we'd need to add role types (like hacker, guard, or watcher) along with tools to keep teams in sync. It brings up design issues too - keeping team tasks separate while avoiding overlap - that'd need solid control over sessions and system resources.

Even so, these joint drills could boost user involvement big time, making the setup feel like real-world cybersecurity training scenarios.

Integration with Real-World Infrastructure (Safely):

To make things feel more real, upcoming updates might try connecting carefully with actual or copied systems. For example, tying the system into a protected copy of a business network can create lifelike drills. Using cloud resources lets teams quickly build short-term digital spaces that mirror live configurations. Specific situations - like hacking smart devices or industrial control systems - may get included later to boost high-level practice.

Each setup like this calls for solid protection so it doesn't touch live systems. Though tough to pull off, having it would boost the platform's worth big time for high-end cybersecurity practice.

Enhanced User Feedback and Learning Analytics:

Because teaching users matters to this platform, tossing in smart tips along with stats on how they're doing would boost what it offers. Down the line, imagine a built-in guide powered by AI that watches what someone does in practice runs, then gives advice shaped just for them. Say a person doesn't block an assault - right after, the tool might point out exactly which safety step would've worked. Bit by bit, number-crunching features could follow their growth, showing shifts and trends right there on screen.

This might need checking simulation records, maybe using smart algorithms to spot trends. Still, keeping info private matters - just learners or approved teachers ought to see how someone's doing.

Continuous Security Validation:

Once it's up and running, the system's safety needs to last. Going forward, building new parts ought to mix security into every step - using automatic checks that run nonstop. Scanning code now and then, checking container setups, plus going over threat models regularly should just be routine stuff. When fresh risks pop up, the way things are built or coded has got to face those

challenges again. To keep protection strong down the line, bringing in outside experts to poke around - or offering rewards for finding flaws - can reveal issues hiding in plain sight.

Compliance and Policy Features:

If schools or groups use this system, they'll have to meet certain security rules along with learning benchmarks. Later updates might offer functions focused on meeting regulations - like logs set up for ISO 27001 checks, or access settings based on roles that fit the NIST/NICE teaching model. Hooking into Learning Management Systems may let teachers monitor who's active and who finishes tasks. Including dashboard utilities for building status reports could help organize cyber safety lessons.

References

- Abhay Talreja. (2025, 10 28). *Spiral Model: Definition, Phases, Advantages & Disadvantages*. Teaching Agile. Retrieved 11 1, 2025, from <https://teachingagile.com/sdlc/models/spiral#:~:text=The%20Spiral%20Model%27s%20greatest%20strength,robust%20framework%20for%20managing%20uncertainty>
- Al-Fayez, R. (n.d.). *SECURITY REQUIREMENTS ENGINEERING: APPLYING SQUARE FRAMEWORK*. SECURITY REQUIREMENTS ENGINEERING: APPLYING SQUARE FRAMEWORK. <https://www.slideshare.net/slideshow/security-requirements-engineering-applying-square-framework/40635137#1>
- The Cyber Range: A Guide*. (n.d.). National Institute of Standards and Technology. Retrieved November 9, 2025, from <https://www.nist.gov/document/cyber-range>
- Cyber Range Project Team and the NICE Community Coordinating Council, Cybersecurity Skills Competitions Community of Interest. (2023). *THE CYBER RANGE*. community coordinating council. https://www.nist.gov/system/files/documents/2023/09/29/The%20Cyber%20Range_A%20Guide.pdf#:~:text=Cyber%20ranges%20are%20interactive%20and,This%20document
- Guest User. (2023, 8 29). *How to Simulate Live Fire Cyber Attacks – Simply Cyber Podcast*. CLODE RANGE. Retrieved 11 1, 2025, from <https://www.cloudrangecyber.com/news/how-to-simulate-live-fire-cyber-attacks-cloud-range-ceo-featured-on-simply-cyber-podcast#:~:text=cybersecurity%20training,solving%20within%20cyber security%20teams>
- Masaryk University. (2022). *KYPO Cyber Range Platform*. Masaryk University. Retrieved 11 9, 2025, from <https://crp.kypo.muni.cz/>
- Spiral model: the risk-minimizing model for software projects*. (n.d.). IONOS. Retrieved November 9, 2025, from <https://www.ionos.com/startupguide/productivity/spiral-model/>
- Talreja, A. (2025, 10 28). *Spiral Model: Definition, Phases, Advantages & Disadvantages*. Spiral Model: Definition, Phases, Advantages & Disadvantages. <https://teachingagile.com/sdlc/models/spiral>

Talreja, A. (2025, 10 28). *Spiral Model: Definition, Phases, Advantages & Disadvantages*. Teaching Agile. Retrieved 11 11, 2025, from <https://www.slideshare.net/slideshow/security-requirements-engineering-applying-square-framework/40635137#:~:text=Image%3A%20SQUARE%20Framework%20Agree%20on,Prioritize%20requirements%20Requirements%20inspection%208>

What is Spiral Model in Software Engineering? (2025, July 11). GeeksforGeeks. Retrieved November 9, 2025, from <https://www.geeksforgeeks.org/software-engineering/software-engineering-spiral-model/>